1.
```
// Exercício Aula 06 - Questão 1
// ANTES de executar: qual é a saída? O problema é detectado na compilação, na execução ou nunca?
// Online: https://onecompiler.com/nodejs  (cole o código inteiro)
// Local:  node q1.js
console.log(0.1 * 3 === 0.3);
console.log(9007199254740993);
```

   A saída esperada é false e 9007199254740992. Quando olhamos para o primeiro console.log não temos problemas, mas quando olhamos para o segundo encontramos problemas de execução, pois o valor inserido no console excede o limite seguro de quantidade de números do JavaScript.

2.
```
# Exercício Aula 06 - Questão 2
# ANTES de executar: qual é a saída?
# Online: https://onecompiler.com/python  (cole o código inteiro)
# Local:  python q2.py
palavra = "maçã"
print(len(palavra), len(palavra.encode()))  # encode() usa UTF-8 por padrão
```

   4, 6, pois quando olhamos a função len, ela nos dá o tamanho da palavra da string, já o método encode() transforma uma string em código UTF-8.

```
// Exercício Aula 06 - Questão 3
// ANTES de executar: qual é a saída? O problema é detectado na compilação, na execução ou nunca?
// Online: https://onecompiler.com/go  (cole o código inteiro)
// Local:  go run q3.go
package main

import "fmt"

func main() {
    var b byte = 255
    b++
    fmt.Println(b)
}
```

3. Quando olhamos para o código vemos um erro de execução, pois o compilador não encontrou nenhum erro antes do programa rodar, e sim temos um erro depois de executarmos o código, isso acontece devido ao overflow.

4.

```java
// Exercício Aula 06 - Questão 4
// ANTES de executar: qual é a saída? O problema é
detectado na compilação, na execução ou nunca?
// Online: https://onecompiler.com/java  (cole o código
inteiro; a classe se chama Main)
// Local:  java Q4.java
public class Main {
    public static void main(String[] args) {
        int[] v = new int[3];
        System.out.println(v[0]);
        System.out.println(v[3]);
    }
}
```

Aqui encontramos um erro de execução, pois o erro é aparente depois da execução, o erro em si consiste em que o vetor int [3] possui 3 elementos, porém seu arrays java seguem o padrão que o início dos arrays começam em zero, então em tese em um array de 3 elementos temos [0, 1, 2], então o valor v[3] não existe na memória.

```rust
4. // Exercício Aula 06 - Questão 5
// ANTES de executar: qual é a saída? O problema é detectado na
compilação, na execução ou nunca?
// Online: https://onecompiler.com/rust  (cole o código inteiro)
// Local:  rustc q5.rs && ./q5
fn main() {
    let s = String::from("oi");
    let t = s;
    println!("{} {}", s, t);
}
```

É um **problema de compilação**. O código sequer chega a gerar um arquivo executável para rodar. Por conta moves do rust, pois, quando implementamos let t = s, peguamos todo o valor contido em s e transferimos para t, tornando o s inválido.

```c
5. /* Exercício Aula 06 - Questão 6
 * ANTES de executar: qual é a saída? O problema é detectado na
compilação, na execução ou nunca?
 * Online: https://onecompiler.com/c  (cole o código inteiro)
 * Local:  gcc q6.c -o q6 && ./q6
 */
#include <stdio.h>

int main(void) {
    union { int i; float f; } u;
    u.f = 1.0f;
```

```
    printf("%d\n", u.i);
    return 0;
}
```

Não temos um erro nem compilação, nem de execução, o programa executa, porém, um problema que podemos encontrar é quando olhamos para printf(“d%\n”,u.i), pois temos declado o valor u.f = 1.0f; e quando printamos o valor u.i ele tenta encontrar um valor que em tese ainda não foi declarado, pegando então um lixo de memória